



5. Neural Networks: Binary Classification

When you need Neural Networks to help you. This means the problem is bigger than you might have thought

Previous Section:

 [4. Neural Networks: The Nine-Step Workflow](#)

Contents:

[1. The IMDB Dataset](#)

[2. Multi-Hot Encoding](#)

[3. Validation and Partial Train Set](#)

[4. Building the Neural Network](#)

[Summary](#)

[References:](#)

So far, when we talked about Machine Learning, things felt clean. You had datasets in tables with rows, columns, features, and labels.

If you wanted to detect spam emails, you might define features like: Number of links; Length of the message; Presence of certain keywords. And from there, the model learns patterns. Simple. Structured. Manageable.

But now, let me ask you something. What if the data doesn't come like that? What if instead of "number of links" or "email length". You are given the **entire email itself**? Not numbers. Not categories. But raw, unstructured **text**. A full sentence. A paragraph. A review.

Something like:

"I hope this letter finds you well... You have invited to our special program... blah blah blah"

Now try to pause for a moment and think like a traditional Machine Learning model. Where are the features? What exactly are you supposed to measure? You could try to force it: Count positive words → Count negative words → Maybe track frequency of certain terms

But very quickly, things start to break down. Because language is messy. The same idea can be written in a thousand different ways. Meaning depends on context. Sarcasm exists. Tone exists. Structure matters.

And suddenly, the idea of manually defining features starts to feel outclassed.

This is where the problem shifts. It's no longer just "classification". It is something deeper. It starts to feel like **understanding**. And that's exactly why problems like this are often grouped under Natural Language Processing (NLP).

But here's the important part—and this is something many people miss, NLP is not separate from Deep Learning. It *leads into it*. Because the moment you stop manually defining features, and start asking the model to learn directly from raw text. You are stepping into the world of **Neural Networks**.

So now, we return to a familiar idea, but in a new setting. Binary Classification. The goal hasn't changed. Yes or No; Positive or Negative; True or False. Still just two outcomes. But the **way we get there** is completely different. Instead of feeding the model carefully engineered features. We give it raw data—and let it figure out what matters.

And to explore this properly, we need a dataset that reflects this reality. Not clean tables. Not handcrafted features. But real human language.

Which brings us to our main case study: The IMDB dataset provided by the TensorFlow library

1. The IMDB Dataset

Now we need something real. Not a toy dataset. Not a perfectly structured table. But actual human language. And this is where the **IMDB Dataset** comes in.

This dataset is widely used in Deep Learning, and it's even built directly into TensorFlow Keras:

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb

# Load the top 10,000 most frequent words
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_w
ords=10000)
```

At first glance, this looks simple. But there's a lot happening behind that one line.

What does "10,000 most frequent words" mean?

Let's start with the part that usually confuses people → `num_words = 10000`

This does **not** mean the dataset only has 10,000 reviews. It means something very different. It controls the **vocabulary size**.

Here's the idea: When working with text, the model cannot read words directly. It needs numbers.

So what Keras does is: It scans the entire dataset. Counts how often each word appears. Ranks them from most frequent to least frequent

Then it keeps only the **top 10,000 most common words**. And assigns each word an integer index:

- "the" → maybe 1
- "movie" → maybe 2
- "good" → maybe 45
- "terrible" → maybe 312

Everything becomes numbers. And anything **outside the top 10,000 words**? It gets ignored or replaced with a special token (like "unknown"). So in reality, each review is no longer text. It becomes a sequence of integers.

What does the original dataset look like?

Before Keras transforms it, the dataset is actually very human.

If you visit [the original source](#), you'll see something like this: A folder for training data and a folder for testing data. Inside each: `pos/` → positive reviews

and `neg/` → negative reviews

Each file is just... a raw text review. For example:

→ `pos/123.txt` → "This movie was fantastic..."

→ `neg/456.txt` → "I didn't like this movie at all..."

So originally, the dataset is: Text → organized into folders → labeled by sentiment.

No numbers. No encoding. Just language.

Now let's look at what happens after loading:

```
print("Train Size:", x_train.shape)
print("Test Size:", x_test.shape)
print("First samples:", len(x_train[0]), "; Label:", y_train[0])
```

Train Size: (25000,)

Test Size: (25000,)

First samples: 218 ; Label: 1

This tells us something very important. There are **25,000 training reviews** and **25,000 testing reviews**. So the dataset is perfectly balanced with 50,000 total reviews → Split evenly into train and test. And each of those reviews already has a label: 1 → Positive and 0 → Negative.

The third line. This is where things get interesting. `x_train[0]` is the **first review** → But remember, it's no longer text. It's a sequence of integers. The output suggests that the first review contains **218 words (tokens)**. Not characters. Not sentences. But **218 encoded words**. And this review is labeled **Positive (1)**

Let's step back and connect everything. Originally: Reviews are raw text → Stored in folders (`pos/`, `neg/`) → Humans can read them easily.

After preprocessing (by Keras): Each word becomes an integer → Each review becomes a sequence of numbers → Only the top 10,000 words are kept.

So the pipeline becomes: `Text → Word ranking → Integer encoding → Model input`

And this is the moment where things truly change. Because now, we are no longer designing features like "number of positive words" → "review length"

Instead, we are feeding the model the **entire review**—just in a numerical form. And from here on, we expect the model to figure out everything else.

2. Multi-Hot Encoding

Now we hit a very important turning point. Because technically we already converted text into numbers. But if you think that means the model can understand it now. Not quite.

Why we still can't use this data (yet)?

Let's go back to what we actually have: Each review looks like this: `[1, 14, 532, 27, 89, ...]`

At first glance, it looks numerical. But those numbers are **not real values**. They are just... **positions**: **Word #1** → **Word #14** → **Word #532**. This is what we call **Label Encoding**. And yes—Keras already did this part for us. But here's the problem:

The model might think: *"Word 532 is bigger than word 14... so it must be more important?"*

Which is completely wrong. These numbers have **no mathematical meaning**. They are just IDs. So if we feed this directly into a Neural Network, we are basically lying to the model.

What about One-Hot Encoding?

You might think: *"Alright... then let's use One-Hot Encoding"*. That works great when you have: One feature → One category per sample. Example: **Color = Red / Blue / Green**

But here? Each review is not **one word**. It's a **sequence of hundreds of words**. So One-Hot Encoding doesn't fit.

The right idea: Multi-Hot Encoding

Instead, we ask a different question: *"Which words appear in this review?"* → Not how many times. Not the order. Just: *"Does this word exist in the review or not?"*

So we represent each review as a vector of size 10,000: One position per word in the vocabulary. And for each position: 1 → word appears; 0 → word does not appear → This is called **Multi-Hot Encoding**.



Multi-Hot Encoding — How It Actually Works

Let's slow this down and make it concrete. Because "Multi-Hot Encoding" sounds abstract, until you actually see what it is doing.

- **Step 1: The Vocabulary (Word → Index Mapping)**

Imagine we already built a vocabulary like this:

```
{1: I, 2: We, 3: You, 4: They, 5: His, 6: Her,
 7: Them, 8: Us,
 9: Good, 10: Great, 11: Excellent, 12: Fantastic,
13: Decent,
14: Amazing, 15: Marvellous, 16: Astonishing, 17:
Beautiful,
18: Bad, 19: Terrible, 20: Horrific, 21: Awful, 2
2: Atrocious, 23: Obnoxious,
24: Movie, 25: Characters, 26: Girl, 27: Boy,
28: Love, 29: Detest, 30: Manifest,
31: God, 32: Intelligent}
```

Now notice something important. This is not "knowledge". It is just a **lookup table**. A way to translate words → positions in a vector.

- **Step 2: Important Idea — Words are not just words**

Before encoding, we already assume something deeper: The model should treat variations of the same word as one concept. So: "love" → "loved" → "loving", are treated as the same feature.

This is usually done through: stemming → lemmatization → or implicit preprocessing rules. So instead of learning 3 separate signals, we compress them into: LOVE (one concept). Same idea applies to verbs: present / past / -ing → normalized into a single representation

- **Step 3: The Sentence**

Now take this review:

┆ "This movie was absolutely amazing... I loved every second of it."

At human level, we read: positive sentiment → excitement → enjoyment. But the model sees something very different.

- **Step 4: Cleaning the Noise**

Words like: "this"; "was"; "every"; "second"; "of"; "it" are mostly ignored. Because they carry **structure**, not **sentiment**. They don't help decide: positive vs negative
So recommender systems and sentiment models usually focus on: keywords; sentiment-bearing words; important nouns and adjectives

- **Step 5: Extracting Meaningful Words**

From the sentence, we keep: "movie"; "amazing"; "love"

Now we map them using our vocabulary: movie → 24; amazing → 14; love → 28

So the review becomes:

```
[24, 14, 28]
```

This is still not learning-ready... but we are close.

- **Step 6: Multi-Hot Encoding in Action**

Now we build a vector of size **32** (vocabulary size):

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Then we activate positions: `index 24 → 1; index 14 → 1; index 28 → 1`

So the final representation becomes:

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
```

- **Step 7: What this really means**

We are no longer storing: words → order → grammar → sentence structure

We are storing only: "Which important concepts appear?"

That's it. This is why it is called **Multi-Hot**:

- "Hot" = 1 (feature exists)
- "Multi" = multiple words can be active at once

Unlike one-hot encoding (one category only), here, many words can be active simultaneously

- **Step 8: The hidden assumption (very important)**

This representation secretly assumes something powerful:

“Word order does not matter — only presence matters.”

So: “I loved the movie, it was amazing”; “The movie was amazing, I loved it”, become identical representations

- **Step 9: Why this is actually useful (even if it feels simple)**

Yes, we lose: grammar → sentence structure → sequence information

But we gain something very important: robustness → simplicity → scalability

For early neural networks (like IMDB sentiment classification), this works surprisingly well because:

- sentiment is often keyword-driven
- words like “amazing”, “terrible”, “great” dominate meaning
- order is less important than presence

You can think of Multi-Hot Encoding like this:

Instead of reading a sentence, the model is checking a checklist of important words

And then asking:

“Which signals are present in this review?”

That’s the entire idea. Simple, but powerful enough to build the first real Neural Network for text classification.

Here’s how we Multi-Hot Encoding with Python:


```

import numpy as np

# Apply Multi-Hot Encoding
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.0

    return results

# Apply to the dataset
x_train = vectorize_sequences(x_train, 10000)
x_test = vectorize_sequences(x_test, 10000)

y_train = np.asarray(y_train).astype('float32')
y_test = np.asarray(y_train).astype('float32')

# Final shape
print('Shape x_train:', x_train.shape)
print('Shape x_test :', x_test.shape)

```

Shape x_train: (25000, 10000)

Shape x_test : (25000, 10000)

Let's break this down slowly. Results creates a matrix with shape: *(number of reviews, 10000 words)*

- So each row = one review
- Each column = one word in the vocabulary

This line is the key: *results[i, sequence] = 1.0*

For each review: Take all the word indices inside it → Set those positions to 1

So if a review contains words. For example *[2, 45, 900]* → Then:

- Index 2 → 1; Index 45 → 1; Index 900 → 1; Everything else stays 0.

Apply to dataset, we notice something very important happens. *x_train* is no longer a list of sequences It becomes a **matrix**.

Each review among 25,000 reviews is: A vector of length **10,000**. Instead of a variable-length sequence (like 218 words). We now have: Fixed-size input → perfect for Neural Networks

We did lose something important: Word order → Frequency → Context; “not good” and “good” look very similar now. But for now, that’s acceptable.

Because our goal is not perfect language understanding. Our goal is: Learn the **first working Neural Network for text classification**. And this representation, is simple, clean, and powerful enough to get us there.

This is the moment where everything connects: We started with raw text → turned it into numbers → and now, we finally have something a Neural Network can actually learn from.

Next step? Build the network itself.

3. Validation and Partial Train Set

Before we jump into training the Neural Network, there’s one more thing we need to get right. And this part? It’s often overlooked by beginners—but it is absolutely critical.

What is Validation Data?

When we train a model, it’s very easy to fall into a trap. The model gets better and better... the training accuracy keeps increasing... And everything looks perfect. Until you test it on new data. And suddenly, it fails.

Because the model didn’t actually **learn**. It memorized.

This is where the **validation set** comes in. It is data the model has **never seen during training updates** But we use it to evaluate the model **during training**

So instead of waiting until the very end, we continuously ask:

“Are you actually learning... or just memorizing?”

Why split from the training set?

In Deep Learning, the common practice is: Take a portion of the training data → Set it aside as validation. Not from the test set. Never the test set. Because: Test set is the final exam; Validation set is the practice checks during learning

Creating the Validation Set

Here’s how we do it:

```
# Create Validation Set
x_val = x_train[:8000]
partial_x_train = x_train[8000:]

y_val = y_train[:8000]
partial_y_train = y_train[8000:]

print("Shape x_val:", x_val.shape)
print("Shape partial_x_train:", partial_x_train.shape)
print("Shape y_val:", y_val.shape)
print("Shape partial_y_train:", partial_y_train.shape)
```

```
Shape x_val: (8000, 10000)
Shape partial_x_train: (17000, 10000)
Shape y_val: (8000,)
Shape partial_y_train: (17000,)
```

Simple slicing, but very intentional. From the original training set: 25,000 samples → We split it into:

- **Validation set** → first 8,000 samples
- **Actual training set** → remaining 17,000 samples

So now: The model trains on **17,000 samples** → And is evaluated on **8,000 unseen samples during training**

What do the output numbers actually mean?

Let's decode this properly.

- `x_val: (8000, 10000)` → 8,000 reviews → Each represented by a 10,000-dimensional vector
- `partial_x_train: (17000, 10000)` → 17,000 reviews used for training → Same feature space
- `y_val: (8000,)` → 8,000 labels (0 or 1)
- `partial_y_train: (17000,)` → 17,000 labels

Now the workflow becomes: Train on `partial_x_train` → After each epoch, evaluate on `x_val` → Monitor performance. And this gives us something powerful: A way to detect **overfitting in real-time**

Think of it like this:

- Training set → where the model learns
- Validation set → where we check if learning is real
- Test set → the final judgment

And this step, even though it looks simple, is what separates: “a model that works” from “a model that actually generalizes”

4. Building the Neural Network

Now comes the part that feels almost... too easy. After everything we've done: **text** → **numbers** → **vectors** → **validation split...**

Training the model is just: **Build** → **Compile** → **Fit**

```
from tensorflow import keras
from tensorflow.keras import layers

# Training the model
hidden_layer1 = layers.Dense(16, activation='relu')
hidden_layer2 = layers.Dense(16, activation='relu')
output_layer = layers.Dense(1, activation='sigmoid')

imdb_model = keras.Sequential([hidden_layer1, hidden_layer
2, output_layer])

# Compile
imdb_model.compile(optimizer='rmsprop',
                    loss='binary_crossentropy', metrics=['ac
curacy'])

# Fit the model
history_imdb = imdb_model.fit(partial_x_train, partial_y_tr
ain,

                               epochs=20, batch_size=256,
                               validation_data=(x_val, y_va

l),

                               verbose=1)
```

```
# Save it, useful for visualization
history = history_imdb.history
```

Let's not overcomplicate this.

- Two hidden layers → each with 16 neurons
- Activation → ReLU (to learn non-linear patterns)
- Output layer → 1 neuron + Sigmoid

And that last part is important: Sigmoid outputs a value between **0 and 1**. Which we interpret as the probability of **positive review**

Nothing entirely new at the remaining of the code—but everything matters.

- **optimizer** → how the model updates itself
- **loss** → how wrong the prediction is
- **accuracy** → how often it's correct

Since this is **binary classification**: `binary_crossentropy` is the correct choice

Finally, we fit the model. And that's it. The model starts learning. Epoch by epoch.

But looking at numbers alone is not enough

You'll see output like: loss decreasing → accuracy increasing. But that's only half the story. So instead of staring at numbers, we **visualize**.

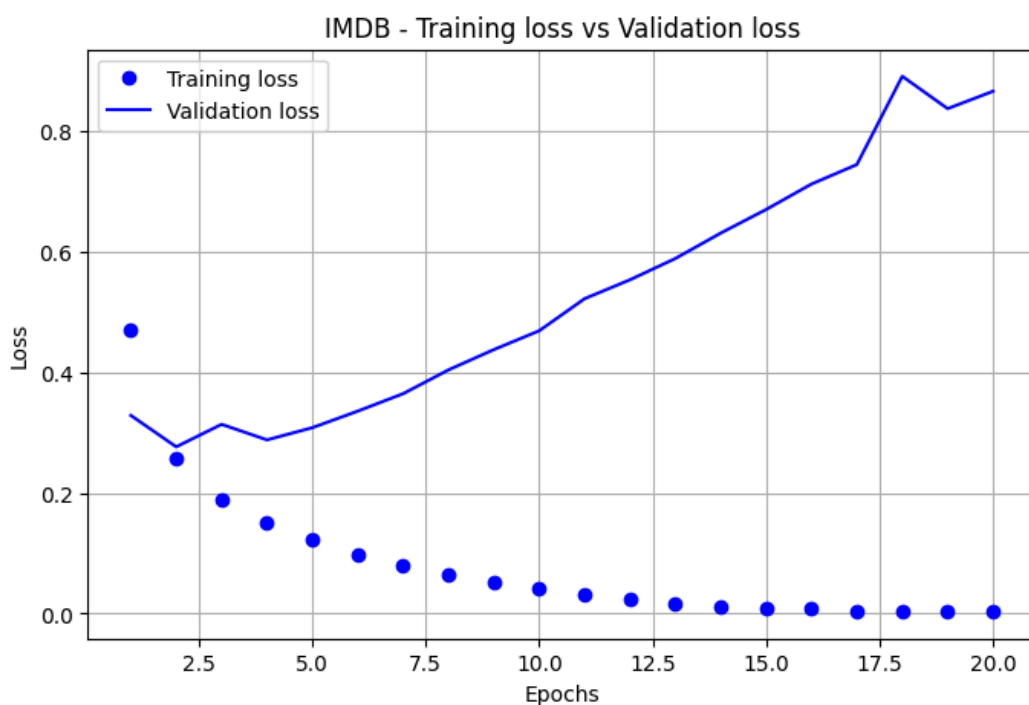
```
import matplotlib.pyplot as plt

# Take loss values from history
loss_values = history['loss']
val_loss_values = history['val_loss']

# create variables
epochs = range(1, len(loss_values) + 1)

# Plotting Training vs Validation Loss
plt.figure(figsize=(8, 5))
```

```
plt.plot(epochs, loss_values, 'bo', label='Training loss')
plt.plot(epochs, val_loss_values, 'b', label='Validation loss')
plt.title('IMDB - Training loss vs Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



Now, this plot tells you everything. It helps answer the question: ***How do we choose the best epoch? Because initially, we just pick an arbitrary number, based on intuition.***

Here's the key idea:

- Training loss will almost always go down
- But validation loss will not

At some point: Validation loss starts increasing. That's the moment the model begins to memorize instead of generalize

So the rule is simple:

Pick the epoch where validation loss is lowest

That's your best model. Not the last epoch. Not the highest accuracy. But the point where the model performs best on unseen data.

This section might feel straightforward. And that's because it is. No new concepts. Just applying everything correctly.

But don't underestimate it. Because this is the first time where: Raw text → becomes predictions; Neural Networks → actually learn meaning

Summary

Nothing fundamentally new was introduced in this whole post. But something important happened: *We trained a full Deep Learning model on real text data*

From raw reviews to binary sentiment predictions. And now that you understand this pipeline, we can push things further.

Binary classification is just the beginning. What happens when: Not just 2 classes but many?

That's where we go next: **Multi-Class Classification**. Using the **Reuters dataset**.

References:

<https://www.kaggle.com/code/karthik7395/binary-classification-using-neural-networks>

<https://www.youtube.com/watch?v=cJ3oqHqRBF0>

Next Section:

 **6. Neural Network: Multi-Class Classification**